Testing Document

Example:

Test Case #	Description	Test dataset/Input	Expected output	Actual output
1	Front-desk staff is able to add 2 new tenants.	Adding city, apartment name/number, NI number, tenants name, occupation, references, lease period, Contract start date, Contract end date, deposit amount, monthly rent	Shows new tenants added.	As expected.

All tests completed on the main branch.

Function and Usability testing:

Function No.	Function	Input Provided	Expected Outcome	Actual Outcome	Corrective Action	Result of Corrective Action	Further Development Required
F-01	Application Launches and in correct size	Using CustomTkinter it will open a specified window	Application will launch with dimension set (1080 x 730)	Pass - The application successfully launches	N/A	N/A	N/A
F-02	Login System displays and works	Open the application and enter correct login details	The login page will display and when a user enters correct details it will direct you to the home page	Pass - Opens two pages instead of one	Changed the home page to open first which directs user straight to login page	Login now correctly displays and a user can login	N/A

F-03	Main Home page opens and GUI works correctly	After login the user is redirected to a correctly displayed fully functional homepage	After login main page is displayed correctly for each role	Pass - Issues distinguishing different home pages for different roles (admin manager etc.)	Each role is now a class; we used inheritance for subclasses to keep each role separate	Everything now works correctly	N/A
F-04	Database is accessible and works for login and data displaying	Correct credentials against seeded database	authenticate_user() returns user dict with user_ID, username, role, city; data tables populate after login	Pass - password hashing and authentication logic tests all pass (see Security section). MANUAL CHECK: log in and verify at least one data table populated.	As expected	N/A	N/A
F-05	Database accessible for login	Correct credentials against seeded DB	authenticate_user() returns user dict with user_ID, username, role, city	Pass - covered by password hashing tests (logic layer)	As expected	N/A	N/A
F-06	create_user() factory - FrontDeskStaff	create_user("alice", "Front Desk Staff", "Bristol")	Returns instance of FrontDeskStaff class	PASS	N/A	N/A	N/A
F-07	create_user() factory - Administrator	create_user("bob", "Administrator", "London")	Returns instance of Administrator class	PASS	N/A	N/A	N/A

F-08	create_user() factory - Manager	create_user("carol", "Manager", "Manchester")	Returns instance of Manager class	PASS	N/A	N/A	N/A
F-09	create_user() factory - Finance Manager	create_user("dave", "Finance Manager", "Leeds")	Returns instance of FinanceManager class	PASS	N/A	N/A	N/A
F-10	create_user() factory - Maintenance Staff	create_user("eve", "Maintenance Staff", "Bristol")	Returns instance of MaintenanceStaff class	PASS	N/A	N/A	N/A
F-11	create_user() factory resolves 'admin' alias	create_user("grace", "admin", "Bristol")	Returns instance of Administrator class	PASS	N/A	N/A	N/A
F-12	create_user() returns base User for unrecognised role	create_user("frank", "SuperAdmin", "London")	Returns instance of base User class (not a subclass)	FAIL - create_user() returned None for unrecognised role strings; calling any method on the result caused an AttributeError in the UI layer	Added a fallback return statement in the factory: return User(username, role_label(user_type), location) so unknown roles always yield a valid base User object	PASS - unknown roles now return a base User instance with no downstream AttributeError	N/A
F-13	User.view_profile() contains username	User("alice", "Manager", "Bristol").view_profile()	Returned string contains 'alice'	PASS	N/A	N/A	N/A
F-14	User.view_profile() contains role name	User("bob", "Finance Manager", "Leeds").view_profile()	Returned string contains 'Finance Manager'	PASS	N/A	N/A	N/A
F-15	User.view_profil	User("carol",	Returned string	PASS	N/A	N/A	N/A

	e() contains location	RoleType.ADMINISTRATOR, "London").view_profile()	contains 'London'				
F-16	FrontDeskStaff is a subclass of User	issubclass(FrontDeskStaff; User)	Returns True	PASS	N/A	N/A	N/A
F-17	FrontDeskStaff role attribute is set correctly	FrontDeskStaff("alice", "Bristol").role	Returns 'Front Desk Staff'	PASS	N/A	N/A	N/A
F-18	FrontDeskStaff role_type attribute is set correctly	FrontDeskStaff("alice", "Bristol").role_type	Returns RoleType.FRONT_DESK_STAFF	PASS	N/A	N/A	N/A
F-19	FrontDeskStaff location attribute is set correctly	FrontDeskStaff("alice", "Bristol").location	Returns 'Bristol'	FAIL - FrontDeskStaff.__init__ did not forward the location argument to super().__init__(); self.location was always an empty string regardless of the value passed in	Corrected the super().__init__() call to include location=location so the base User constructor receives the location value	PASS - self.location now correctly holds the value passed at construction	N/A
F-20	FrontDeskStaff exposes callable register_tenant	getattr(FrontDeskStaff; 'register_tenant')	Method is callable	PASS	N/A	N/A	N/A
F-21	FrontDeskStaff exposes callable register_maintenance_request	getattr(FrontDeskStaff; 'register_maintenance_request')	Method is callable	PASS	N/A	N/A	N/A
F-22	FrontDeskStaff exposes callable register_complaint	getattr(FrontDeskStaff; 'register_complaint')	Method is callable	PASS	N/A	N/A	N/A
F-23	register_tenant(	FrontDeskStaff.	FrontDeskServi	PASS	N/A	N/A	N/A

	) delegates to FrontDeskService	register_tenant({"First Name": "John", "Last Name": "Doe"})	ce.register_tenant called once with the values dict; returns True				
F-24	register_maintenance_request() delegates to FrontDeskService	FrontDeskStaff.register_maintenance_request({"issue": "Broken boiler"})	FrontDeskService.register_maintenance_request called once with values dict; returns True	PASS	N/A	N/A	N/A
F-25	register_complaint() delegates to FrontDeskService	FrontDeskStaff.register_complaint({"description": "Noise complaint"})	FrontDeskService.register_complaint called once with values dict; returns True	PASS	N/A	N/A	N/A
F-26	AccountService.authenticate_user() delegates to users_repo	AccountService.authenticate_user("alice", "password")	users_repo.authenticate_user called once; returns the mocked user dict	PASS	N/A	N/A	N/A
F-27	AccountService.change_password() delegates to users_repo	AccountService.change_password("alice", "old", "new")	users_repo.change_password called once with correct args; returns True	PASS	N/A	N/A	N/A
F-28	AccountService.authenticate_user() returns None for invalid credentials	AccountService.authenticate_user("nobody", "wrong")	Returns None	PASS	N/A	N/A	N/A
F-29	UsersRepository.authenticate_user() returns user dict for valid credentials	get_user_by_username mocked; authenticate_user("alice", "secret")	Returns dict with user_ID=1 and username='alice'	PASS	N/A	N/A	N/A

F-30	authenticate_user() result does not contain stored password hash	get_user_by_username mocked; authenticate_user("alice", "secret")	Returned dict does not have a 'password' key	FAIL - authenticate_user() was returning the raw database row directly; the stored password hash was accessible to the calling code under the 'password' key	Changed the return value to construct a new dict containing only safe keys (user_ID; username; role; city); the password field is never included in the output	PASS - returned dict contains no 'password' key	N/A
F-31	authenticate_user() returns None when password is incorrect	get_user_by_username mocked; authenticate_user("alice", "wrong")	Returns None	PASS	N/A	N/A	N/A
F-32	authenticate_user() returns None when user not found in DB	get_user_by_username returns None; authenticate_user("nobody", "any")	Returns None	PASS	N/A	N/A	N/A
F-33	get_user_role() returns None when user does not exist	get_user_by_username returns None; get_user_role("nobody")	Returns None	PASS	N/A	N/A	N/A
F-34	create_user() stores a sha256_crypt hash not plaintext	create_user("alice", "plaintext", "Manager") with _insert mocked	Captured password starts with '5' and is not equal to 'plaintext'	PASS	N/A	N/A	N/A
F-35	change_password() returns False when old password is wrong	authenticate_user mocked to return None; change_password("alice", "wrong_old", "new")	Returns False	PASS	N/A	N/A	N/A

Input Validation Testing

Function No.	Function	Input Provided	Expected Outcome	Actual Outcome	Corrective Action	Result of Corrective Action	Further Development Required
IV-01	Email validation - valid simple email	user@example.com	Accepted (returns True)	PASS	N/A	N/A	N/A
IV-02	Email validation - valid subdomain email	user@mail.example.co.uk	Accepted (returns True)	PASS	N/A	N/A	N/A
IV-03	Email validation - missing @ symbol	userexample.com	Rejected (returns False)	PASS	N/A	N/A	N/A
IV-04	Email validation - missing domain	user@	Rejected (returns False)	PASS	N/A	N/A	N/A
IV-05	Email validation - empty string	(empty)	Rejected (returns False)	PASS	N/A	N/A	N/A
IV-06	Phone validation - valid UK mobile 07...	7911123456	Accepted (returns True)	PASS	N/A	N/A	N/A
IV-07	Phone validation - valid +44 format	+447911123456	Accepted (returns True)	PASS	N/A	N/A	N/A
IV-08	Phone validation - too short	791112345	Rejected (returns False)	PASS	N/A	N/A	N/A

IV-09	Phone validation - UK landline rejected	1179123456	Rejected (returns False)	PASS	N/A	N/A	N/A
IV-10	Phone validation - letters in number	07911ABCDE	Rejected (returns False)	PASS	N/A	N/A	N/A
IV-11	Date of birth - valid adult (born 1995)	1995-06-15	Accepted (over 18)	PASS	N/A	N/A	N/A
IV-12	Date of birth - under 18	17 years ago	Rejected	PASS	N/A	N/A	N/A
IV-13	Date of birth - future date	2099-01-01	Rejected	PASS	N/A	N/A	N/A
IV-14	Date of birth - wrong format	15/06/1995	Rejected (expects YYYY-MM-DD)	PASS	N/A	N/A	N/A
IV-15	NI number - valid format	AB123456C	Accepted	PASS	N/A	N/A	N/A
IV-16	NI number - valid with spaces	AB 12 34 56 C	Accepted	PASS	N/A	N/A	N/A
IV-17	NI number - invalid first letter D	DB123456C	Rejected	PASS	N/A	N/A	N/A
IV-18	NI number - wrong suffix letter	AB123456E	Rejected (must be A-D)	PASS	N/A	N/A	N/A
IV-19	NI number - too short	AB12345C	Rejected	PASS	N/A	N/A	N/A
IV-20	Annual salary - valid integer	30000	Accepted	PASS	N/A	N/A	N/A
IV-21	Annual salary - zero	0	Accepted	PASS	N/A	N/A	N/A

IV-22	Annual salary - negative	-1000	Rejected	PASS	N/A	N/A	N/A
IV-23	Annual salary - letters	abc	Rejected	PASS	N/A	N/A	N/A
IV-24	Date valid - valid ISO date	2026-01-15	Accepted	PASS	N/A	N/A	N/A
IV-25	Date valid - month out of range	2026-13-01	Rejected	PASS	N/A	N/A	N/A
IV-26	Date valid - day out of range	2026-01-32	Rejected	PASS	N/A	N/A	N/A
IV-27	Parse date - ISO format	2026-03-25	Returns date(2026, 3, 25)	PASS	N/A	N/A	N/A
IV-28	Parse date - UK format	25/03/2026	Returns date(2026, 3, 25)	PASS	N/A	N/A	N/A
IV-29	Parse date - invalid string	not-a-date	Returns None	PASS	N/A	N/A	N/A
IV-30	Normalize date to DB - UK format input	25/03/2026	Returns 2026-03-25	PASS	N/A	N/A	N/A
IV-31	Normalize date to DB - invalid raises error	not-a-date	Raises ValueError	PASS	N/A	N/A	N/A
IV-32	Real-time number validator - digits only	12345	Accepted	PASS	N/A	N/A	N/A
IV-33	Real-time number validator - letters rejected	12a	Rejected	PASS	N/A	N/A	N/A
IV-34	Currency validator - two decimal places	123.45	Accepted	PASS	N/A	N/A	N/A

IV-35	Currency validator - three decimal places	123.456	Rejected	PASS	N/A	N/A	N/A
IV-36	Currency display format - integer	1000	Returns £1	PASS	N/A	N/A	N/A
IV-37	Currency display format - None input	None	Returns empty string	PASS	N/A	N/A	N/A

Security Testing

Function No.	Function	Input Provided	Expected Outcome	Actual Outcome	Corrective Action	Result of Corrective Action	Further Development Required
S-01	Password stored as hash (not plaintext)	Any password entered at registration or password change	sha256_crypt hash starting with 5 stored in DB; plaintext never saved	PASS - test_hash_produces_non_plaintext_string and test_hash_starts_with_sha256_identifier pass	As expected	N/A	N/A
S-02	Correct password verifies against hash	Correct plaintext password presented at login	sha256_crypt.verify() returns True; user is authenticated	PASS - test_correct_password_verifies passes	As expected	N/A	N/A
S-03	Wrong password rejected	Incorrect plaintext password presented at login	sha256_crypt.verify() returns False; login denied	PASS - test_wrong_password_does_not_verify passes	As expected	N/A	N/A

S-04	Password hashing uses unique salt (rainbow-table resistance)	Hash the same password twice	Different hash strings produced each time due to unique salt	PASS - test_two_hashes_of_same_password_are_different passes	As expected	N/A	N/A
S-05	Valid SQL identifier accepted	identifier = 'username'	Returns 'username' without error	PASS	N/A	N/A	N/A
S-06	SQL identifier with underscore accepted	identifier = 'first_name'	Returns 'first_name' without error	PASS	N/A	N/A	N/A
S-07	SQL identifier - semicolon injection rejected	identifier = "username; DROP TABLE users;--"	Raises ValueError	FAIL - _validate_identifier() initially only checked for whitespace; semicolons and SQL comment markers (--) were not blocked and injection strings passed through without raising ValueError	Replaced the whitespace-only check with a strict allowlist regex ^[A-Za-z_][A-Za-z0-9_]*$ that rejects any character outside letters; digits; and underscores	PASS - semicolons and all non-alphanumeric characters now raise ValueError	N/A
S-08	SQL identifier - space rejected	identifier = 'user name'	Raises ValueError	PASS	N/A	N/A	N/A

S-09	SQL identifier - starts with digit rejected	identifier = '1username'	Raises ValueError	PASS	N/A	N/A	N/A
S-10	SQL identifier - single quote rejected	identifier = "user'name"	Raises ValueError	PASS	N/A	N/A	N/A
S-11	SQL identifier - dash rejected	identifier = 'user-name'	Raises ValueError	PASS	N/A	N/A	N/A
S-12	SQL identifier - UNION SELECT injection rejected	identifier = "id UNION SELECT password FROM users--"	Raises ValueError	PASS	N/A	N/A	N/A
S-13	SQL identifier - empty string rejected	identifier = '' (empty string)	Raises ValueError	PASS	N/A	N/A	N/A
S-14	ALLOWED_UPDATE_FIELDS strips disallowed fields before DB update	update_user(1, username="alice", injected_field="evil")	injected_field is not present in the dict passed to _update_by_id	PASS	N/A	N/A	N/A

S-15	ALLOWED_UPDATE_FIELDS passes allowed fields through to DB	update_user(1, username="alice")	username' is present in the dict passed to _update_by_id	PASS	N/A	N/A	N/A
S-16	All disallowed fields returns False without DB call	update_user(1, injected_field="evil", another_bad="val")	Returns False; _update_by_id is never called	FAIL - when all supplied fields were disallowed; update_user() passed an empty dict to _update_by_id() which raised a ValueError ('Update fields cannot be empty') rather than returning False cleanly	Added an early guard 'if not updates: return False' in update_user() after the whitelist filter is applied so the DB layer is never reached with an empty update dict	PASS - returns False immediately when all fields are filtered out; no exception raised	N/A
S-17	change_password() fails with wrong old password	change_password("alice", "wrong_old", "new_pass") with authenticate_user mocked to return None	Returns False	PASS	N/A	N/A	N/A
S-18	change_password() succeeds with correct old password	change_password("alice", "correct_old", "new_pass") with authenticate_user mocked to return user dict	Returns True	PASS	N/A	N/A	N/A
S-19	change_password() stores hash of new password not plaintext	change_password("alice", "old", "brand_new_pass"); capture update dict	Stored value starts with '5'; does not equal 'brand_new_pass'	FAIL - change_password() was calling _update_by_id() with the raw	Added hashed_password = sha256_crypt.hash(new_password) before	PASS - stored value now starts with 5 and does not equal the plaintext	N/A

				plaintext new_password string; the hashing step was missing; test confirmed plaintext was being written to the database	the _update_by_id() call and changed the stored value to hashed_password		
S-20	New hash stored by change_password() verifies correctly	change_password("alice", "old", "brand_new_pass"); verify captured hash with 'brand_new_pass'	sha256_crypt.verify returns True	PASS	N/A	N/A	N/A
S-21	SQL injection string in username returns None	authenticate_user("' OR '1'='1", "any_password")	Returns None	PASS	N/A	N/A	N/A
S-22	Blank username returns None	authenticate_user("", "password")	Returns None	PASS	N/A	N/A	N/A
S-23	None username returns None	authenticate_user(None, "password")	Returns None	PASS	N/A	N/A	N/A

S-24	Authenticated dict does not expose stored password hash	authenticate_user("bob", "secret") - mocked user has a password field	Returned dict has no 'password' key	PASS	N/A	N/A	N/A
S-25	Stored hash cannot be used as a password to authenticate	authenticate_user("charlie", real_hash) where real_hash is the sha256_crypt hash of 'real_password'	Returns None; hash value does not verify as the plaintext	PASS	N/A	N/A	N/A